

¡APOCO te necesita!

Juan Manuel Salazar Salazar 20251020026

Sebastián Adrián Guzmán Gómez 20251020079

Andres Camilo Ramos Rojas 20242020005

1 Resumen

2 Introducción

El proyecto APOCO (Asociación de Políticos Corruptos) consiste en una aplicación desarrollada en el lenguaje Java, la cual integra conceptos fundamentales de estructuras de datos y algoritmos de ordenamiento. El sistema permite la gestión y clasificación de registros de individuos denominados "Políticos" y "Hampones", utilizando listas enlazadas implementadas de forma manual para garantizar un control total sobre el manejo de la memoria y la lógica de punteros. El objetivo primordial de la herramienta es realizar pruebas de rendimiento comparativas entre diversos algoritmos de ordenamiento, evaluando su eficiencia en términos de tiempo de ejecución y cantidad de iteraciones necesarias para organizar datos de forma descendente basándose en criterios económicos.

3 Descripción del Programa

La aplicación se presenta como una plataforma interactiva donde el usuario puede configurar la generación de datos aleatorios. Se permite definir la cantidad de registros a simular o, en el caso específico del auditorio de hampones, establecer una disposición de filas y columnas que determina el tamaño de la población. Una vez recibidos los parámetros, el sistema genera automáticamente los objetos con nombres y cifras monetarias aleatorias. Internamente, el programa clona la lista original para someterla a cuatro procesos de ordenamiento independientes. Durante estas pruebas, se

capturan métricas precisas mediante el uso de nanosegundos y contadores internos de ciclos. Finalmente, la plataforma presenta los resultados en dos formatos: una tabla detallada con los registros organizados y un podio visual que destaca los 3 algoritmos con mejor desempeño temporal.

4 Estructura y Flujo

Se ha adoptado el patrón de arquitectura Modelo-Vista-Controlador (MVC) para asegurar la modularidad y escalabilidad del código:

- **Modelo:** Contiene las entidades de negocio como Persona, Corrupto y Hampon, además de las estructuras de datos (ListaEnlazadaSimple) y los objetos de transferencia de datos (ResultadoPrueba) que empaquetan las métricas de rendimiento.
- **Vista:** Implementada mediante páginas JSP y HTML, se encarga de la captura de parámetros de entrada y la representación gráfica de los datos procesados, utilizando etiquetas JSTL para dinamizar el contenido.
- **Controlador:** Representado por los servlets CorruptoServlet y HamponesServlet, los cuales actúan como intermediarios. Estos reciben las peticiones del usuario, invocan los servicios de generación y ordenamiento, y redirigen los resultados a la vista correspondiente.

El flujo operativo se inicia con la interacción del usuario en los formularios de configuración. El controlador procesa la solicitud, genera la lista de datos y ejecuta la competencia de algoritmos a través de un servicio especializado. Tras completar las pruebas, los datos se envían a la interfaz de usuario para su visualización.

5 Algoritmos de Ordenamiento

Se han implementado cuatro estrategias clásicas de ordenamiento, cada una cumpliendo con el contrato definido por la interfaz funcional EstrategiaOrdenamiento, lo que permite el intercambio dinámico de algoritmos:

1. **Ordenamiento Burbuja (Bubble Sort):** Se basa en la comparación de elementos adyacentes y el intercambio de sus datos y el orden es incorrecto. Este proceso se repite hasta que el elemento mayor "flota" hacia su posición final. Utiliza ciclos anidados y el método ordenar().
2. **Ordenamiento por Inserción (Insertion Sort):** Construye la lista ordenada de manera progresiva. Toma cada nodo de la lista original e invoca el método auxiliar insertarEnOrden() para ubicarlo en la posición exacta dentro de una sublista ya clasificada.
3. **Ordenamiento por Mezcla (Merge Sort):** Emplea la técnica de divide y vencerás. Divide la lista en mitades hasta llegar a elementos individuales mediante el método mergeSortRecurso(), para luego unirlos de forma ordenada usando la función fusionar().
4. **Ordenamiento Rápido (Quick Sort):** Selecciona un elemento como pivote y organiza la lista de modo que los elementos mayores queden a un lado y los menores al otro a través del método particionar(). Posteriormente, se aplica de forma recursiva a las sublistas resultantes.

6 Resultados y análisis

6.1 Entorno de Pruebas

Dado que las métricas de tiempo de ejecución de los algoritmos de ordenamiento son altamente dependientes de la capacidad de procesamiento de la máquina, a continuación se detallan las especificaciones del hardware utilizado para realizar las simulaciones:

- **Procesador:** 12th Gen Intel(R) Core(TM) i7-1255U (1.70 GHz)
- **Memoria RAM instalada:** 16.0 GB
- **Tipo de sistema:** Sistema operativo de 64 bits, procesador x64

6.2 Políticos Corruptos

Para la sección de "Políticos Corruptos", se realizaron pruebas de estrés con distintas densidades de entrada para medir la eficiencia de los algoritmos

en términos de iteraciones y tiempo. Los resultados permiten clasificar los métodos según su rendimiento térmico, cuyo orden se presenta en la Figura 1.



Figure 1: Ranking de algoritmos según tiempo de ejecución.

La tabla comparativa resultante es la base para el análisis de eficiencia algorítmica bajo distintos volúmenes de datos. Con el objetivo de obtener datos estadísticamente significativos, cada configuración se sometió a cinco pruebas con arreglos generados de forma aleatoria, promediando los resultados para reducir la varianza y normalizar el error experimental.

	Bubble	Insertion	Merge	Quick
Prueba 1	4797	2631	540	648
Prueba 2	4830	2480	541	607
Prueba 3	4935	2723	542	601
Prueba 4	4872	2476	534	701
Prueba 5	4929	2637	534	661
Promedio	4873	2589	538	644

(a) Número de iteraciones

	Bubble	Insertion	Merge	Quick
Prueba 1	0.34	0.012	0.014	0.011
Prueba 2	0.166	0.011	0.017	0.01
Prueba 3	0.32	0.011	0.012	0.01
Prueba 4	0.032	0.01	0.014	0.01
Prueba 5	0.032	0.011	0.014	0.01
Promedio	0.178	0.011	0.0142	0.0102

(b) Tiempo de ejecución (ms)

Figure 2: Resultados de las tablas con N= 100 elementos

	Bubble	Insertion	Merge	Quick
Prueba 1	124560	62279	3862	4525
Prueba 2	124254	61871	3868	5215
Prueba 3	124744	62415	3866	4991
Prueba 4	124285	63418	3851	4643
Prueba 5	124009	64535	3844	4411
Promedio	124370	62904	3858	4757

(a) Número de iteraciones

	Bubble	Insertion	Merge	Quick
Prueba 1	1.022	0.399	0.479	0.193
Prueba 2	1.375	0.353	0.406	0.216
Prueba 3	1.047	0.353	0.215	0.139
Prueba 4	1.015	0.363	0.146	0.141
Prueba 5	0.973	0.39	0.333	0.163
Promedio	1.0864	0.3716	0.3158	0.1704

(b) Tiempo de ejecución (ms)

Figure 3: Resultados de las tablas con N= 500 elementos

	Bubble	Insertion	Merge	Quick
Prueba 1	499122	245020	8696	10296
Prueba 2	499200	249627	8706	10557
Prueba 3	499149	244392	8700	10458
Prueba 4	497789	254691	8703	9916
Prueba 5	499310	260722	8715	10607
Promedio	498914	250890	8704	10367

(a) Número de iteraciones

	Bubble	Insertion	Merge	Quick
Prueba 1	3.984	1.987	0.481	0.28
Prueba 2	4.611	2.063	0.345	0.307
Prueba 3	5.698	2.319	0.432	0.432
Prueba 4	3.983	2.027	0.336	0.305
Prueba 5	3.971	2.361	0.453	0.302
Promedio	4.4494	2.1514	0.4094	0.3252

(b) Tiempo de ejecución (ms)

Figure 4: Resultados de las tablas con N= 1000 elementos

	Bubble	Insertion	Merge	Quick
Prueba 1	12495085	6328088	55166	73334
Prueba 2	12489372	6211866	55201	72291
Prueba 3	12495420	6151971	55233	67773
Prueba 4	12491505	6297673	55212	70149
Prueba 5	12496510	6180512	55340	69977
Promedio	12493578	6234022	55230	70705

(a) Número de iteraciones

	Bubble	Insertion	Merge	Quick
Prueba 1	111.55	100.481	2.454	2.025
Prueba 2	115.928	91.251	2.387	2.395
Prueba 3	111.707	89.435	2.559	1.931
Prueba 4	173.15	88.75	2.099	2.182
Prueba 5	104.345	80.393	2.09	2.18
Promedio	123.336	90.062	2.3178	2.1426

(b) Tiempo de ejecución (ms)

Figure 5: Resultados de las tablas con N= 5000 elementos

	Bubble	Insertion	Merge	Quick
Prueba 1	49973264	24683278	120528	158765
Prueba 2	49992920	24938113	120394	161448
Prueba 3	49989950	25040893	120375	151844
Prueba 4	49994565	24829450	120379	155199
Prueba 5	49988672	25165656	120453	154934
Promedio	49987874	24931478	120426	156438

(a) Número de iteraciones

	Bubble	Insertion	Merge	Quick
Prueba 1	513.726	411.971	5.161	4.535
Prueba 2	592.137	413.791	5.588	4.733
Prueba 3	609.533	373.778	5.461	4.315
Prueba 4	552.034	419.51	5.783	4.821
Prueba 5	510.82	420.542	5.338	4.623
Promedio	555.65	407.9184	5.4662	4.6054

(b) Tiempo de ejecución (ms)

Figure 6: Resultados de las tablas con $N= 10000$ elementos

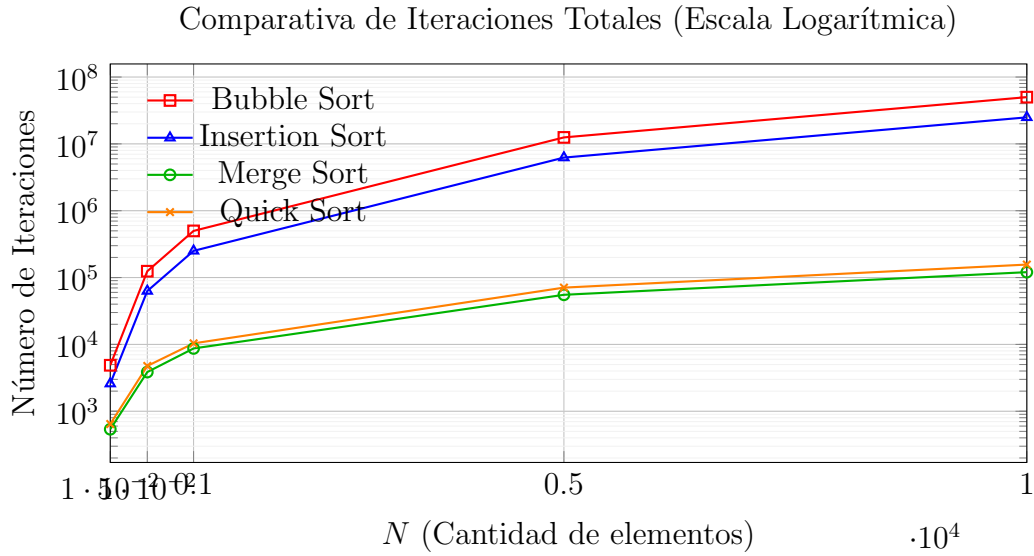


Figure 7: Crecimiento del esfuerzo computacional (iteraciones) en función del tamaño de entrada N [cite: 1, 4].

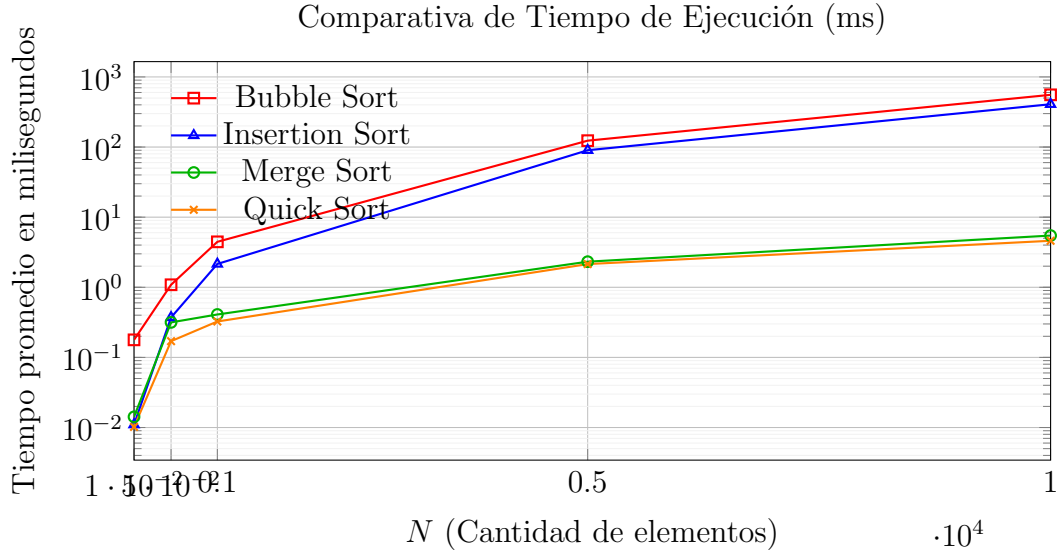


Figure 8: Relación entre el tiempo de ejecución y el tamaño de la muestra para los cuatro algoritmos[cite: 1, 4].

6.3 Hampones y Auditorios

Para la sección de "Hampones", la Asociación de Políticos Corruptos (APOCO) solicitó organizar a los miembros en un auditorio de dimensiones $k \times m$. Es fundamental destacar que, aunque la disposición estética en la vista se presenta en filas y columnas (organizadas por edad y dinero robado), la estructura de datos subyacente sigue siendo una lista. Por ello, se observa que los resultados de eficiencia son prácticamente idénticos a los obtenidos en la sección de "Políticos Corruptos", ya que el esfuerzo computacional depende del número total de elementos N .

Se confirma que los resultados son equivalentes a los de las tablas de políticos siempre que el producto de las filas y columnas ($k \times m$) coincida con el valor de N evaluado. A continuación, se presentan los resultados promediados de las 5 pruebas realizadas para cada densidad de entrada.

	Bubble	Insertion	Merge	Quick
Prueba 1	4949	2446	540	785
Prueba 2	4895	2527	542	638
Prueba 3	4740	2610	545	595
Prueba 4	4949	2551	532	604
Prueba 5	4940	2363	543	597
Promedio	4895	2499	540	644

(a) Número de iteraciones (N=100)

	Bubble	Insertion	Merge	Quick
Prueba 1	0.278	0.994	0.028	0.747
Prueba 2	0.666	0.043	0.029	0.075
Prueba 3	0.318	0.021	0.024	0.022
Prueba 4	0.632	0.022	0.030	0.025
Prueba 5	0.273	0.019	0.024	0.024
Promedio	0.4334	0.2198	0.0270	0.1786

(b) Tiempo de ejecución ms (N=100)

Figure 9: Resultados experimentales para Hampones (N=100).

	Bubble	Insertion	Merge	Quick
Prueba 1	124705	62961	3839	4798
Prueba 2	124254	61214	3843	4735
Prueba 3	124474	65104	3856	5143
Prueba 4	124285	61960	3851	4456
Prueba 5	124747	63617	3851	4638
Promedio	124493	62971	3848	4754

(a) Número de iteraciones (N=500)

	Bubble	Insertion	Merge	Quick
Prueba 1	2.723	0.458	0.167	0.153
Prueba 2	3.370	0.442	0.169	0.198
Prueba 3	3.225	0.460	0.167	0.201
Prueba 4	2.913	0.449	0.167	0.147
Prueba 5	1.688	0.449	0.166	0.154
Promedio	2.7838	0.4516	0.1672	0.1706

(b) Tiempo de ejecución ms (N=500)

Figure 10: Resultados experimentales para Hampones (N=500).

	Bubble	Insertion	Merge	Quick
Prueba 1	499434	241204	8725	10731
Prueba 2	496574	248082	8697	10617
Prueba 3	499395	249819	8682	10446
Prueba 4	498069	256057	8715	10968
Prueba 5	499455	245556	8685	12744
Promedio	498585	248144	8701	11101

(a) Número de iteraciones (N=1000)

	Bubble	Insertion	Merge	Quick
Prueba 1	8.428	2.274	0.364	0.361
Prueba 2	7.493	2.202	0.365	0.335
Prueba 3	9.324	2.173	0.383	0.641
Prueba 4	8.897	1.837	0.487	0.492
Prueba 5	8.886	2.156	0.510	0.417
Promedio	8.6056	2.1284	0.4218	0.4492

(b) Tiempo de ejecución ms (N=1000)

Figure 11: Resultados experimentales para Hampones (N=1000).

	Bubble	Insertion	Merge	Quick
Prueba 1	12489750	6259793	55237	69103
Prueba 2	12481569	6298003	55140	72171
Prueba 3	12497464	6293481	55248	69515
Prueba 4	12495609	6154729	55240	65577
Prueba 5	12496905	6301332	55266	70499
Promedio	12492259	6261468	55226	69373

(a) Número de iteraciones (N=5000)

	Bubble	Insertion	Merge	Quick
Prueba 1	237.665	93.555	2.409	2.320
Prueba 2	219.174	95.984	2.298	2.058
Prueba 3	215.912	96.497	2.404	2.118
Prueba 4	223.130	94.733	2.587	2.066
Prueba 5	218.375	95.627	2.431	2.691
Promedio	222.8512	95.2792	2.4258	2.2506

(b) Tiempo de ejecución ms (N=5000)

Figure 12: Resultados experimentales para Hampones (N=5000).

	Bubble	Insertion	Merge	Quick
Prueba 1	49990629	25162528	120504	149572
Prueba 2	49992515	24776327	120455	152024
Prueba 3	49984704	25028366	120499	158873
Prueba 4	49978347	25285857	120433	147263
Prueba 5	49987860	24968606	120445	154268
Promedio	49986811	25044337	120467	152400

(a) Número de iteraciones (N=10000)

	Bubble	Insertion	Merge	Quick
Prueba 1	996.194	444.108	5.298	4.871
Prueba 2	995.632	434.293	5.223	5.023
Prueba 3	937.441	436.969	4.903	4.749
Prueba 4	999.123	445.039	5.554	4.651
Prueba 5	1004.423	436.910	5.727	4.724
Promedio	986.5626	439.4638	5.3410	4.8036

(b) Tiempo de ejecución ms (N=10000)

Figure 13: Resultados experimentales para Hampones (N=10000).

Para las siguientes gráficas, se utiliza una escala logarítmica en el eje Y para permitir la comparación simultánea de algoritmos con crecimientos muy disparos.

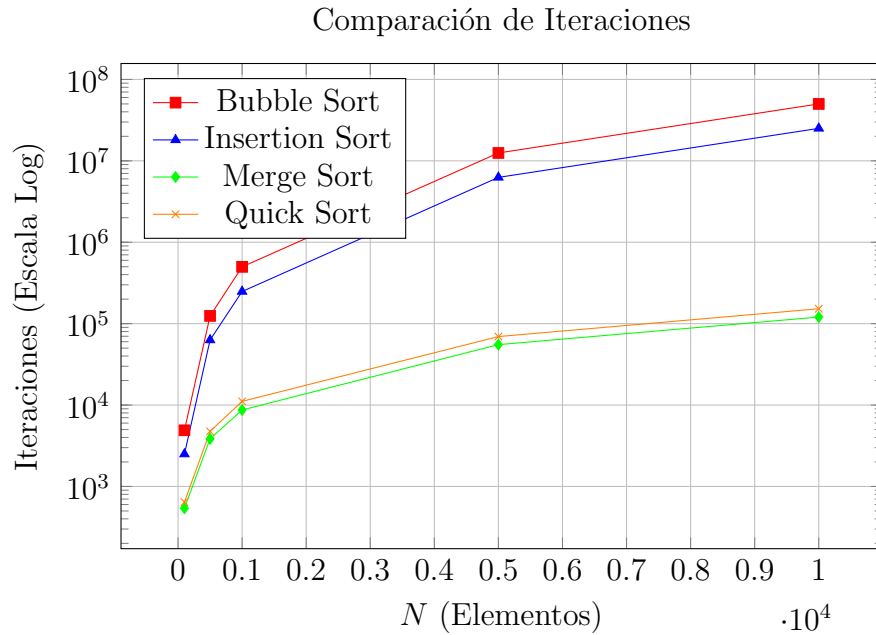


Figure 14: Crecimiento del número de iteraciones en función de N .

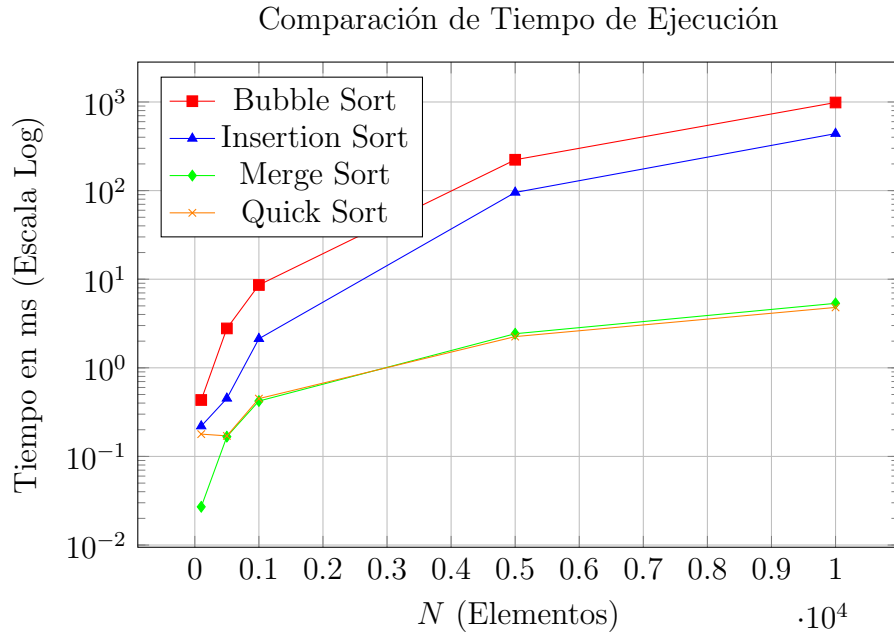


Figure 15: Tiempo de ejecución promedio en milisegundos en función de N .

7 Conclusiones

Al analizar las tablas y la variación de los datos, queda claro que los algoritmos de tipo Divide y Vencerás juegan en otra liga. En prácticamente todos los escenarios, QuickSort y MergeSort demostraron ser los más eficientes, procesando la información de manera mucho más ágil cuando la carga de trabajo aumenta.

Por otro lado, notamos que para cantidades de datos bajas no hay cambios sustanciales en los tiempos de ejecución. En estos casos, la diferencia es tan mínima que no vale la pena complicarse eligiendo uno sobre otro; cualquier método cumple la tarea en un tiempo imperceptible.

El ordenamiento de burbuja es, sin duda, el menos viable si hablamos de potencia bruta. Su rendimiento es el más flojo, necesitando millones de iteraciones para organizar una lista que los algoritmos de Divide y Vencerás resuelven con una fracción del esfuerzo y la memoria.

A pesar de su lentitud, la "Burbuja" se sigue usando bastante por lo fácil que es de implementar. En contextos donde la rapidez del código no es crítica

o para listas muy pequeñas, su sencillez compensa su falta de eficiencia.

En definitiva, la elección del algoritmo debe depender directamente del tamaño del problema. Si buscamos escalabilidad y velocidad en proyectos grandes, los métodos avanzados son obligatorios; pero para tareas rápidas y sencillas, lo clásico sigue teniendo su lugar.